

Name of Experiment: Performance Analysis of Spatial Filters for Salt-and-Pepper Noise Removal Using PSNR

Introduction: In these experiments, a 512×512 grayscale image was corrupted with salt-and-pepper noise of different noise levels. To remove the noise, two spatial filtering techniques, Average Filter and Median Filter, were applied using a 5×5 kernel. The filtered images were then compared visually, and in Task-1 PSNR values were also calculated to evaluate image quality. The main objective of these experiments was to analyze the effectiveness of both filters in removing salt-and-pepper noise from grayscale images.

Task-1: Removal of 20% Salt-and-Pepper Noise Using Average and Median Filters.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('/content/5226523_orig.jpg', cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (512, 512))

def add_sp_noise(image, prob=0.05):
    noisy = image.copy()
    rand = np.random.rand(*image.shape)
    noisy[rand < prob/2] = 0
    noisy[rand > 1 - prob/2] = 255
    return noisy

noisy_img = add_sp_noise(img, 0.20)

avg_filtered = cv2.blur(noisy_img, (5, 5))
median_filtered = cv2.medianBlur(noisy_img, 5)

def psnr(original, processed):
    mse = np.mean((original - processed) ** 2)
    if mse == 0:
        return 100
    return 10 * np.log10((255 ** 2) / mse)

psnr_noisy = psnr(img, noisy_img)
psnr_avg = psnr(img, avg_filtered)
psnr_median = psnr(img, median_filtered)

print(psnr_noisy)
print(psnr_avg)
print(psnr_median)

cv2.imwrite('original_image.png', img)
cv2.imwrite('noisy_image.png', noisy_img)
cv2.imwrite('average_filtered_image.png', avg_filtered)
cv2.imwrite('median_filtered_image.png', median_filtered)

plt.figure(figsize=(10,8))
plt.subplot(2,2,1)
plt.imshow(img, cmap='gray')
plt.title('Original')
plt.axis('off')

plt.subplot(2,2,2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy (20% SP Noise)')
plt.axis('off')

plt.subplot(2,2,3)
plt.imshow(avg_filtered, cmap='gray')
plt.title('Average Filter (5x5)')
plt.axis('off')

plt.subplot(2,2,4)
```

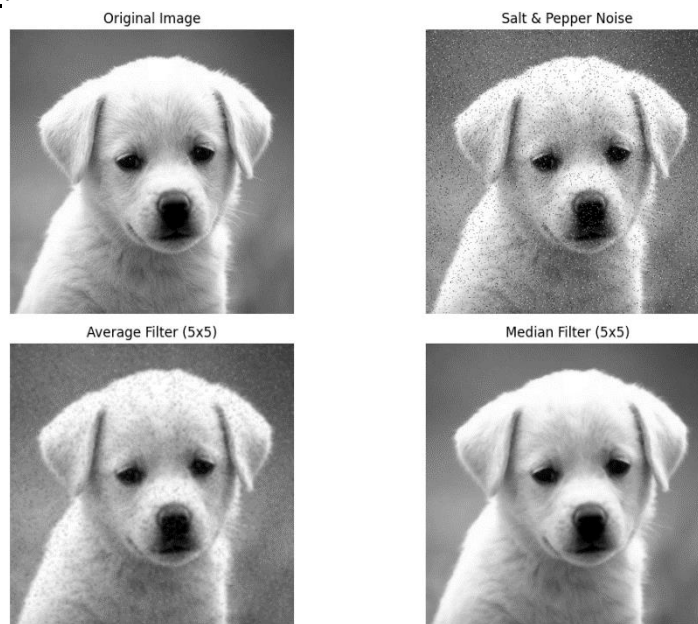
```
plt.imshow(median_filtered, cmap='gray')
plt.title('Median Filter (5x5)')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Output Data-1:

PSNR of Average Filter (5x5): 29.63299196267338
 PSNR of Median Filter (5x5): 36.08607333596774

Output Image-1:



Task-2: Filtering of 5% Salt-and-Pepper Noise Using Average and Median Filters.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('/content/5226523_orig.jpg', cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (512, 512))

def add_sp_noise(image, prob=0.05):
    noisy = image.copy()
    rand = np.random.rand(*image.shape)
    noisy[rand < prob/2] = 0
    noisy[rand > 1 - prob/2] = 255
    return noisy

noisy_img = add_sp_noise(img, 0.05)

avg_filtered = cv2.blur(noisy_img, (5, 5))
median_filtered = cv2.medianBlur(noisy_img, 5)

cv2.imwrite('original.png', img)
cv2.imwrite('noisy.png', noisy_img)
cv2.imwrite('average_filtered.png', avg_filtered)
cv2.imwrite('median_filtered.png', median_filtered)

plt.figure(figsize=(10,8))
plt.subplot(2,2,1)
```

```

10,8))
plt.subplot(2,2,1)
plt.imshow(img, cmap='gray')
plt.title('Original')
plt.axis('off')

plt.subplot(2,2,2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy (5% SP Noise)')
plt.axis('off')

plt.subplot(2,2,3)
plt.imshow(avg_filtered, cmap='gray')
plt.title('Average Filter (5x5)')
plt.axis('off')

plt.subplot(2,2,4)
plt.imshow(median_filtered, cmap='gray')
plt.title('Median Filter (5x5)')
plt.axis('off')

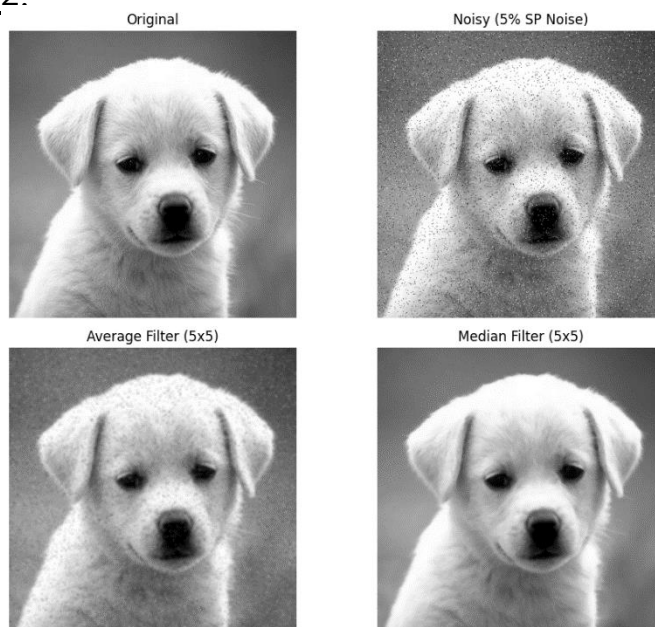
plt.tight_layout()
plt.show()

```

Output Data-2:

PSNR (3x3 Average): 15.993544230567924
 PSNR (5x5 Average): 17.656034882067352
 PSNR (7x7 Average): 18.214337997621193

Output Image-2:



Task-3: Comparison of Average and Median Filters for Noise Removal.

```
img = cv2.imread("/content/5226523_orig.jpg", cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (512, 512))

def add_salt_pepper_noise(image, amount=0.05):
    noisy = np.copy(image)

    num_salt = int(np.ceil(amount * image.size * 0.5))
    coords = [np.random.randint(0, i - 1, num_salt) for i in image.shape]
    noisy[coords[0], coords[1]] = 255

    num_pepper = int(np.ceil(amount * image.size * 0.5))
    coords = [np.random.randint(0, i - 1, num_pepper) for i in image.shape]
    noisy[coords[0], coords[1]] = 0

    return noisy

noisy_img = add_salt_pepper_noise(img)

def harmonic_filter(image, k=5):
    image = image.astype(np.float64) + 1
    pad = k // 2
    padded = np.pad(image, pad, mode='reflect')
    output = np.zeros_like(image)

    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            region = padded[i:i+k, j:j+k]
            output[i, j] = (k * k) / np.sum(1.0 / region)

    return np.clip(output, 0, 255).astype(np.uint8)

def geometric_filter(image, k=5):
    image = image.astype(np.float64) + 1
    pad = k // 2
    padded = np.pad(image, pad, mode='reflect')
    output = np.zeros_like(image)

    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            region = padded[i:i+k, j:j+k]
            output[i, j] = gmean(region.flatten())

    return np.clip(output, 0, 255).astype(np.uint8)

harmonic_img = harmonic_filter(noisy_img, 5)
geometric_img = geometric_filter(noisy_img, 5)

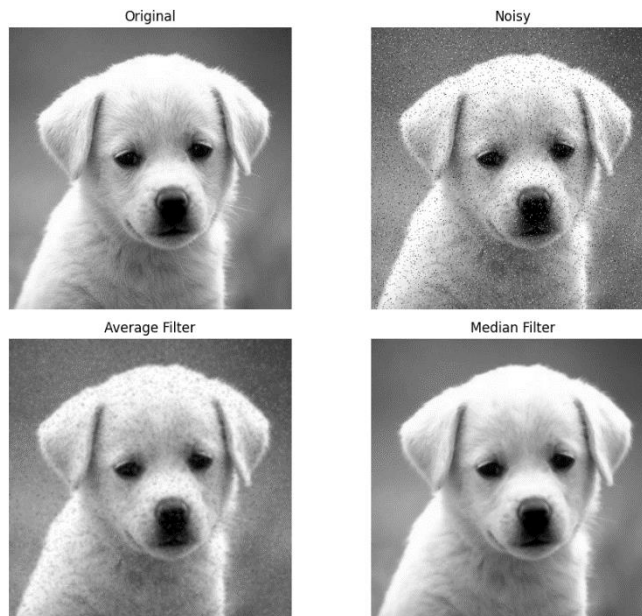
psnr_harmonic = psnr(img, harmonic_img)
psnr_geometric = psnr(img, geometric_img)

print("PSNR (Harmonic Mean Filter):", psnr_harmonic)
print("PSNR (Geometric Mean Filter):", psnr_geometric)
```

Output Data:

```
PSNR (Harmonic Mean Filter): 8.009643013549812
PSNR (Geometric Mean Filter): 19.068784737704586
```

Output Image:



Discussion: The noisy images contain random black and white pixels caused by salt-and-pepper noise. The Average Filter reduces noise by smoothing neighboring pixels, but it also causes blurring and loss of image details. On the other hand, the Median Filter removes noise more effectively while preserving edges and important image features. As the noise level increases, the difference in performance becomes more noticeable. The results show that the Median Filter performs better than the Average Filter for removing salt-and-pepper noise from grayscale images.